LAPORAN PRAKTIKUM

POSTTEST 4

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Nuril Akmal 2509106074

Kelas B2'25

PROGRAM STUDI INFORMATIKA

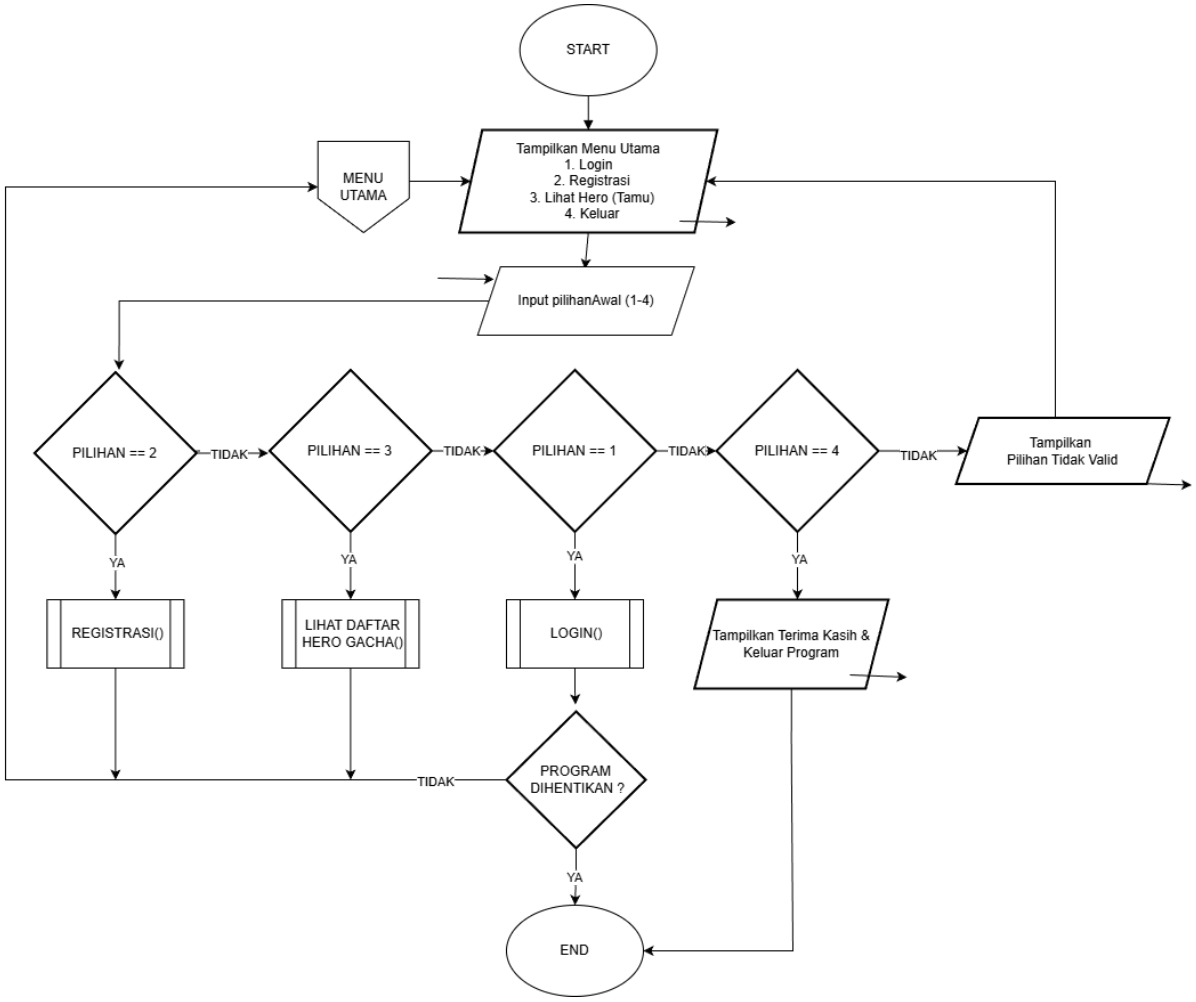
UNIVERSITAS MULAWARMAN

SAMARINDA

2025

1. **Flowchart**

1.1 Menu Utama



Gambar 1.1 Flowchart Menu Utama

Penjelasan:

Inisialisasi: Program dimulai dengan menampilkan menu dan meminta input pengguna (angka 1–4).

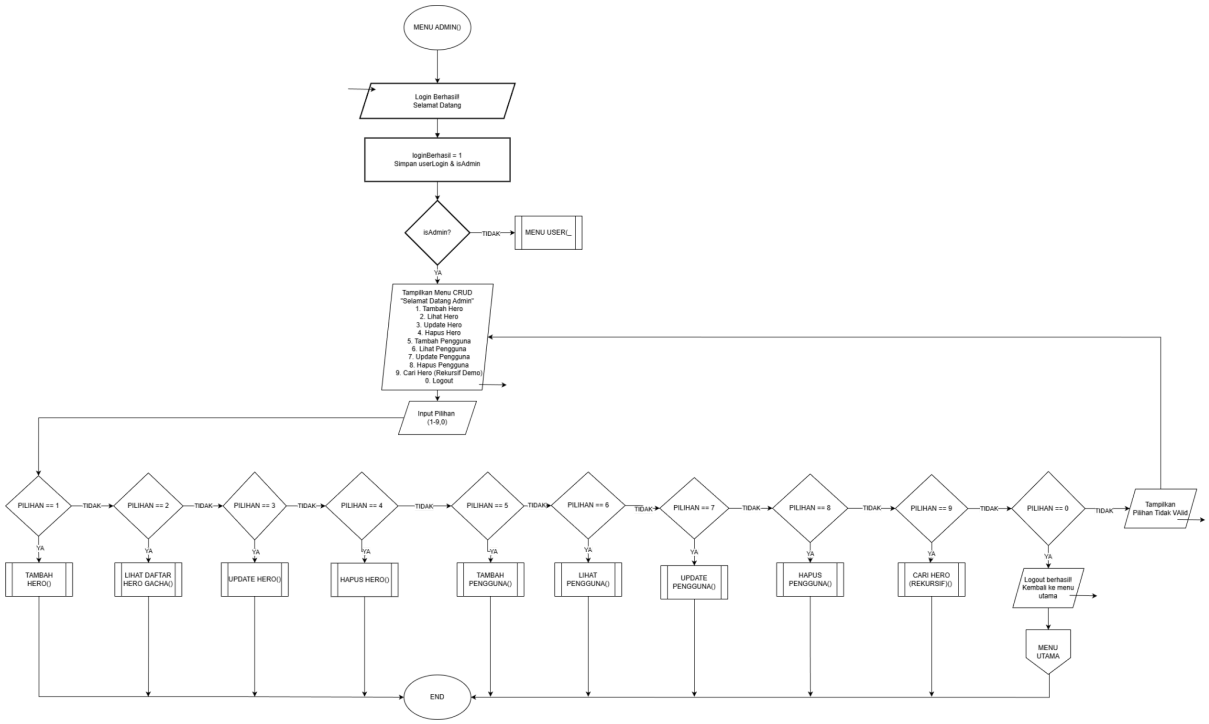
Percabangan Logika: Program memvalidasi input untuk menentukan prosedur yang dijalankan:

- Opsi 1 (Login): Menjalankan fungsi LOGIN().
- Opsi 2 (Registrasi): Menjalankan fungsi REGISTRASI().
- Opsi 3 (Lihat Hero): Menjalankan fungsi LIHAT DAFTAR HERO GACHA().
- Opsi 4 (Keluar): Menampilkan pesan penutup dan mengakhiri sesi.

Penanganan Kesalahan: Jika input tidak sesuai (di luar 1–4), program menampilkan pesan "Pilihan Tidak Valid" dan kembali ke menu utama.

Terminasi: Program akan berakhir (END) jika pengguna memilih opsi keluar atau memenuhi kondisi penghentian program setelah proses login.

1.2 Menu Admin()



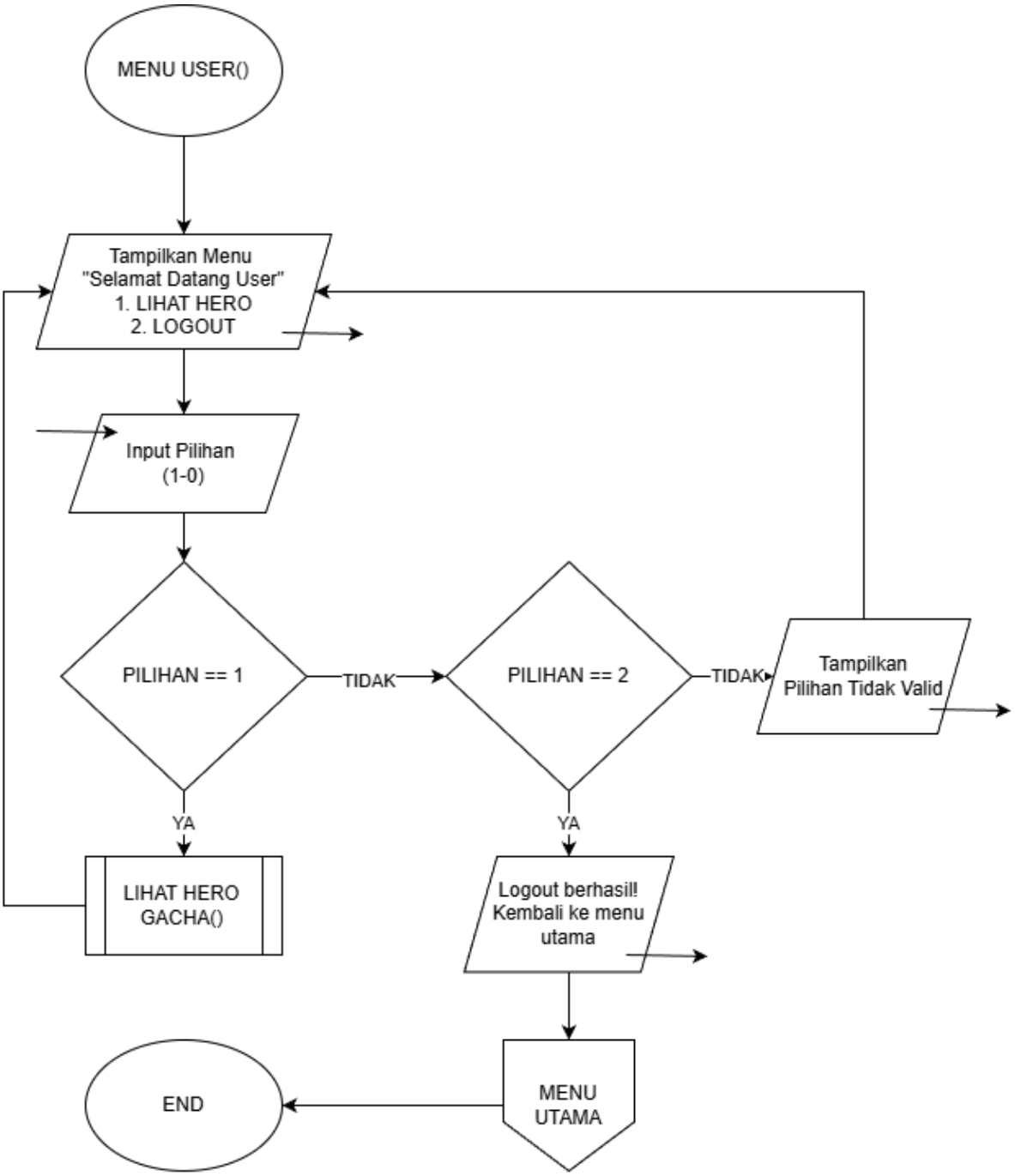
Gambar 1.2 Flowchart Menu Admin

Penjelasan:

Flowchart ini menjelaskan prosedur khusus bagi pengguna dengan hak akses Admin untuk mengelola data sistem (CRUD).

- Validasi Akses: Program dimulai dengan pengecekan status IsAdmin?. Jika bukan admin, diarahkan ke MENU USER(), jika benar, menampilkan Menu CRUD.
- Opsi Manajemen Data: Admin dapat memilih 10 opsi interaksi (angka 1–9 dan 0):
    - Manajemen Hero (1-4): Tambah, Lihat, Update, dan Hapus data hero.
    - Manajemen Pengguna (5-8): Tambah, Lihat, Update, dan Hapus data pengguna.
    - Pencarian (9): Melakukan fungsi CARI HERO secara rekursif.
    - Logout (0): Kembali ke menu utama.
- Eksekusi & Validasi: Setiap pilihan memanggil sub-proses (fungsi) yang sesuai. Jika input di luar angka yang tersedia, sistem menampilkan pesan "Pilihan Tidak Valid".
- Terminasi: Alur berakhir pada titik END setelah aksi dipilih atau setelah admin melakukan logout.

1.3 Menu User()



Gambar 1.3 Flowchart Menu User

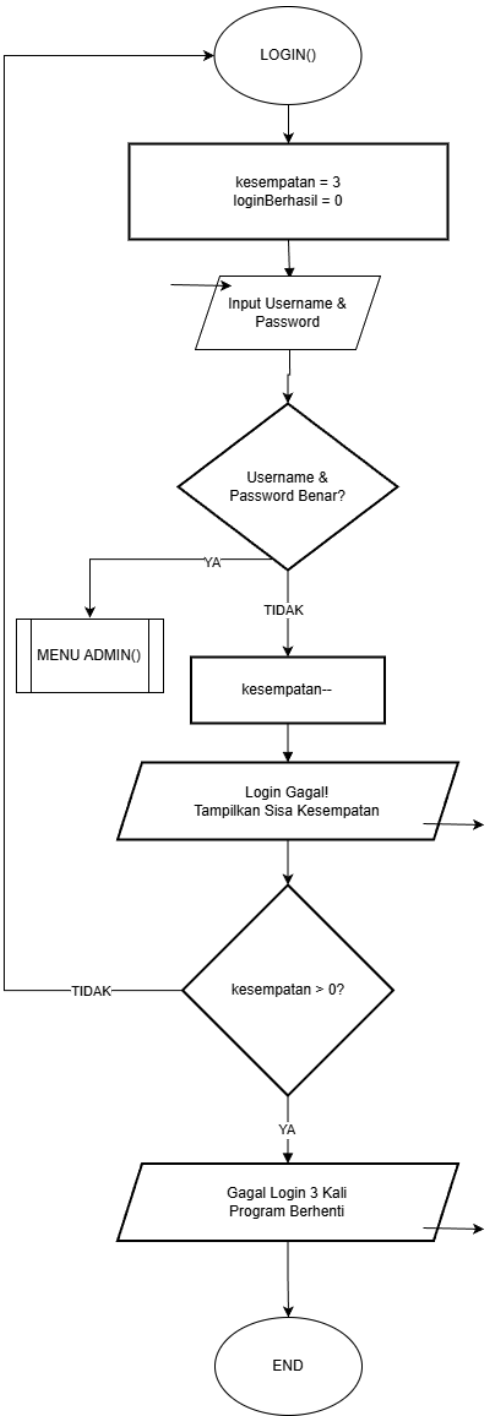
Penjelasan:

Flowchart ini menggambarkan navigasi sederhana untuk pengguna yang sudah login (non-admin):

- Inisialisasi: Program menampilkan menu "Selamat Datang User" dengan dua opsi utama.
- Percabangan Logika:

    - Pilihan 1 (Lihat Hero): Menjalankan prosedur LIHAT HERO GACHA() dan mengakhiri sesi (END).
    - Pilihan 2 (Logout): Menampilkan pesan logout berhasil dan mengarahkan kembali ke MENU UTAMA.
- Validasi: Jika input tidak valid (bukan 1 atau 2), sistem menampilkan pesan "Pilihan Tidak Valid" dan kembali meminta input pengguna.

1.4 Login()



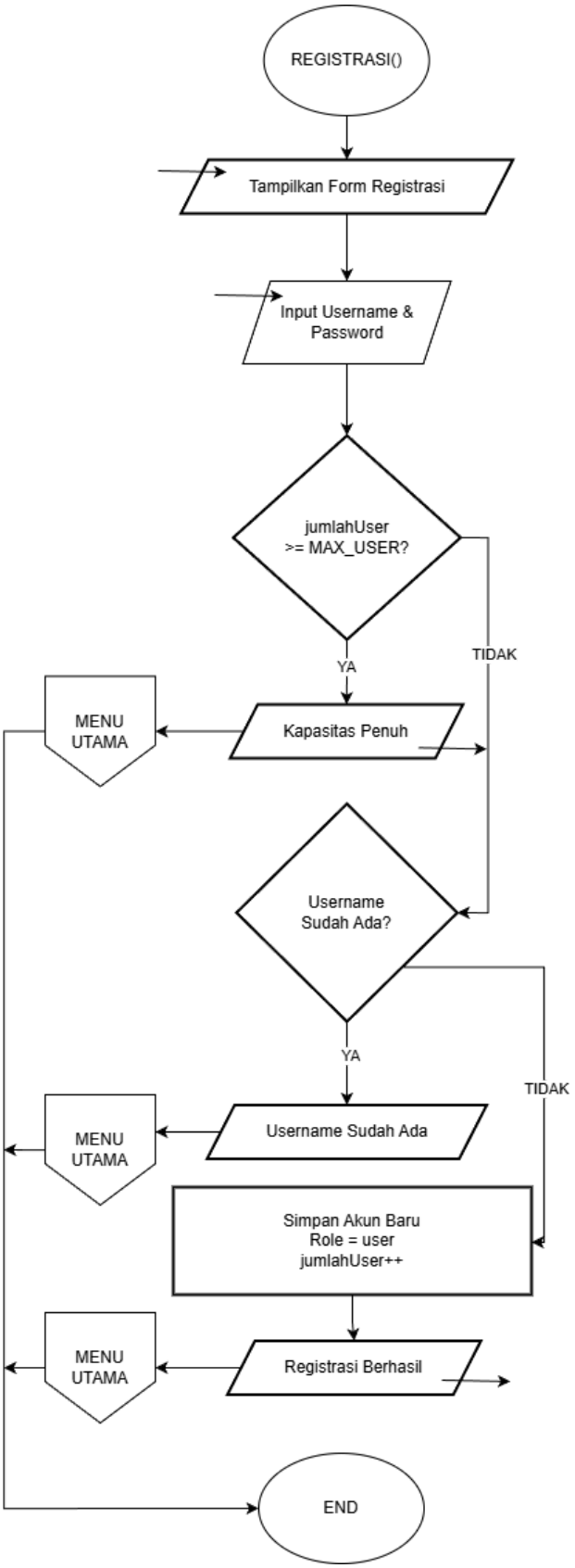
Gambar 1.4 Flowchart Login

Penjelasan:

Flowchart ini menggambarkan prosedur autentikasi pengguna dengan sistem keamanan pembatasan percobaan:

- Inisialisasi: Program menetapkan variabel kesempatan = 3 dan loginBerhasil = 0.
- Input & Verifikasi: Pengguna memasukkan *Username* dan *Password*. Sistem kemudian mengecek validitas data tersebut.
- Kondisi Berhasil: Jika data benar, pengguna langsung diarahkan ke sub-proses MENU ADMIN().
- Kondisi Gagal & Pengurangan: Jika data salah, variabel kesempatan berkurang 1, dan sistem menampilkan pesan kegagalan serta sisa kesempatan yang ada.
- Limit Percobaan:
    - Jika kesempatan > 0, pengguna dapat mengulang proses input (looping kembali ke atas).
    - Jika kesempatan habis (0), program menampilkan pesan "Gagal Login 3 Kali Program Berhenti" dan alur berakhir (END).

1.5 Registrasi()



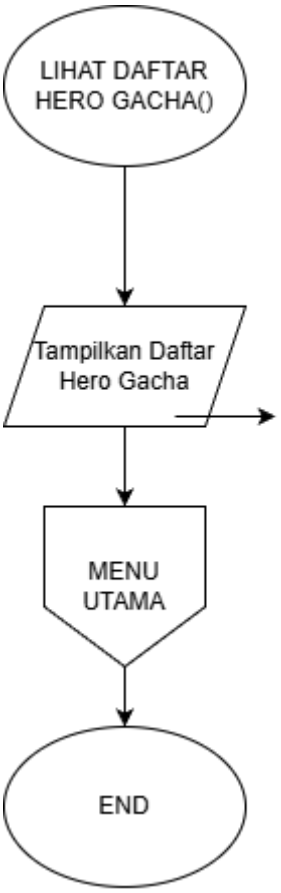
Gambar 1.5 Flowchart Registrasi

Penjelasan:

Flowchart ini menggambarkan prosedur pendaftaran akun baru dengan beberapa tahap validasi keamanan dan kapasitas:

- Inisialisasi & Input: Program menampilkan form registrasi dan meminta pengguna memasukkan *Username* serta *Password*.
- Validasi Kapasitas: Sistem mengecek apakah jumlah pengguna saat ini sudah mencapai batas maksimal (jumlahUser >= MAX_USER). Jika penuh, program menampilkan pesan "Kapasitas Penuh" dan kembali ke MENU UTAMA.
- Validasi Duplikasi: Jika kapasitas masih tersedia, sistem mengecek apakah *Username* sudah terdaftar. Jika sudah ada, tampil pesan "Username Sudah Ada" dan kembali ke MENU UTAMA.
- Penyimpanan Data: Jika *Username* unik, sistem menyimpan akun baru dengan peran sebagai user, lalu menambah hitungan total pengguna (jumlahUser++).
- Konfirmasi & Selesai: Program menampilkan pesan "Registrasi Berhasil" dan alur berakhir (END) atau kembali ke MENU UTAMA.

1.6 Lihat Daftar Hero Gacha()



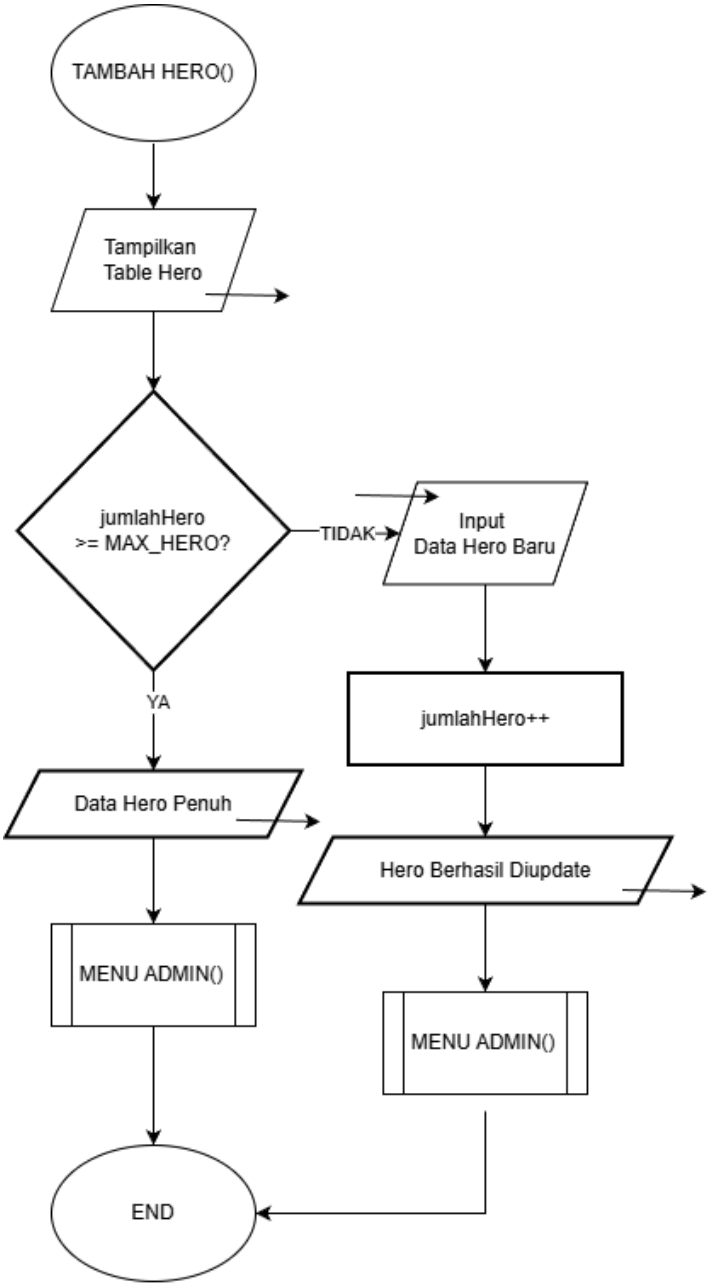
Gambar 1.6 Flowchart Lihat Hero

Penjelasan:

Flowchart ini menggambarkan prosedur sederhana untuk menampilkan informasi data kepada pengguna:

- Inisialisasi: Program memanggil fungsi LIHAT DAFTAR HERO GACHA().
- Output: Sistem menampilkan seluruh daftar hero yang tersedia dalam database ke layar pengguna.
- Navigasi & Selesai: Setelah data ditampilkan, alur diarahkan kembali ke MENU UTAMA dan prosedur berakhir (END).

1.7 Tambah Hero()



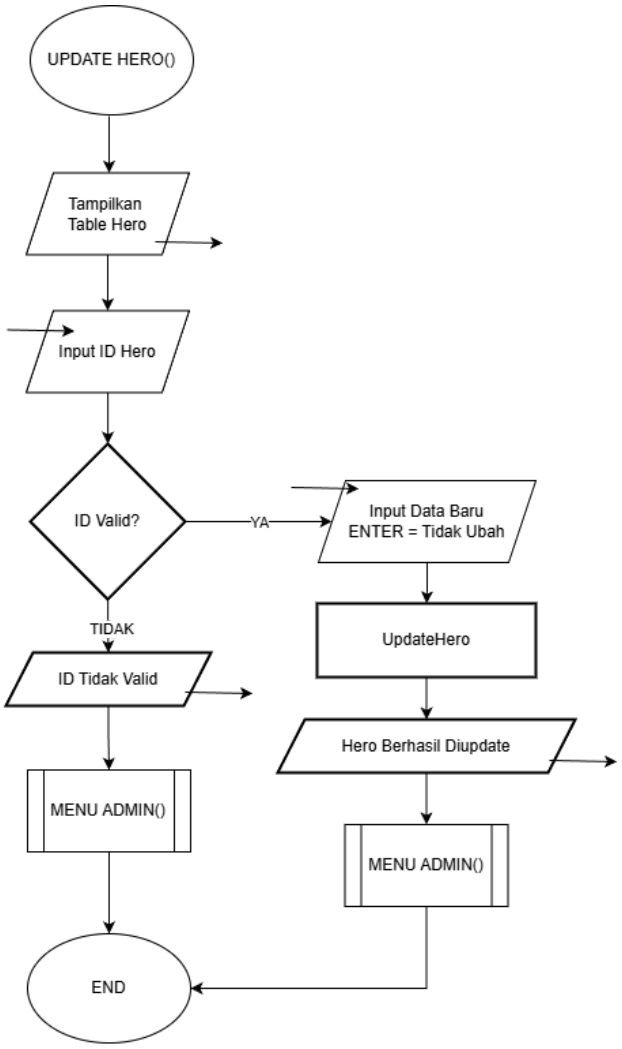
Gambar 1.7 Flowchart Tambah Hero

Penjelasan:

Flowchart ini merinci proses penambahan data hero baru ke dalam sistem oleh Admin:

- Inisialisasi: Program menampilkan tabel hero yang ada saat ini sebelum memulai proses input.
- Validasi Kapasitas: Sistem memeriksa apakah jumlah hero sudah mencapai batas maksimal (jumlahHero >= MAX_HERO).
    - Jika Penuh: Menampilkan pesan "Data Hero Penuh" dan kembali ke MENU ADMIN().
- Proses Input: Jika masih ada ruang, admin memasukkan data hero baru.
- Pembaruan Data: Sistem menambah hitungan total hero (jumlahHero++) dan memberikan konfirmasi "Hero Berhasil Diupdate".
- Selesai: Alur diakhiri dengan kembali ke MENU ADMIN() dan prosedur ditutup (END).

1.8 Update Hero()



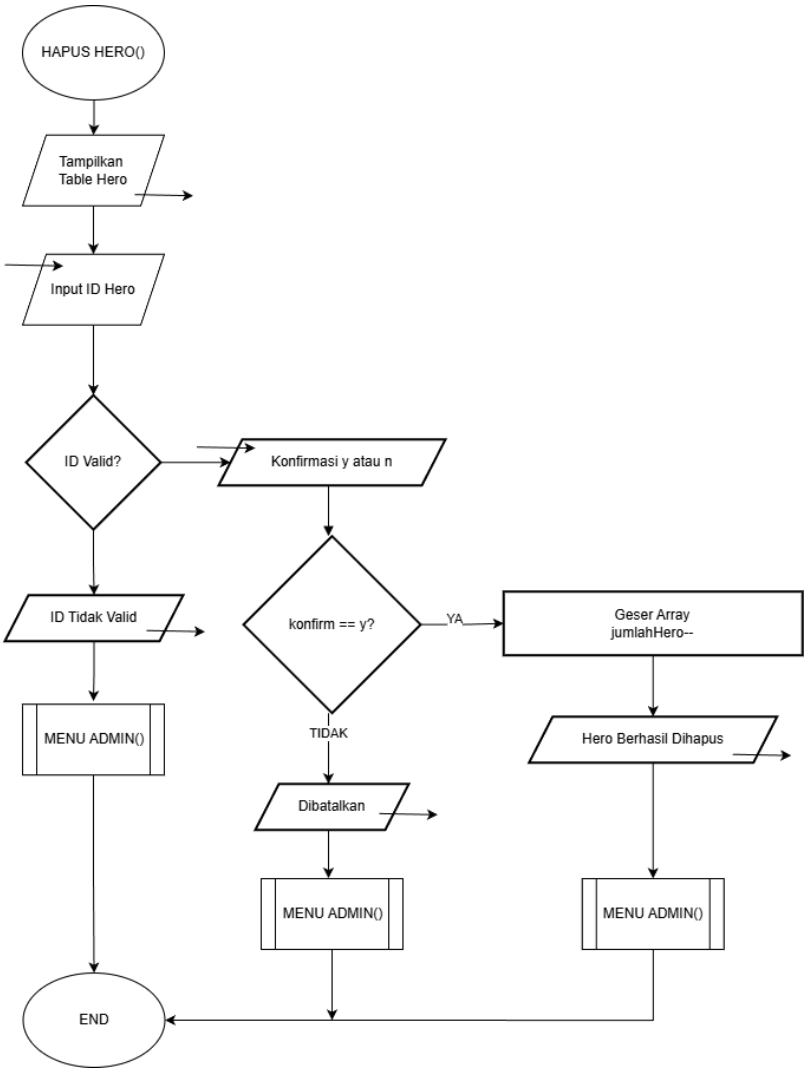
Gambar 1.8 Flowchart Update Hero

Penjelasan:

Flowchart ini merinci prosedur untuk mengubah data hero yang sudah ada di dalam sistem:

- Inisialisasi & Pencarian: Program menampilkan tabel hero dan meminta Admin memasukkan ID Hero yang ingin diubah.
- Validasi ID: Sistem mengecek apakah ID tersebut tersedia. Jika TIDAK, muncul pesan "ID Tidak Valid" dan kembali ke MENU ADMIN().
- Proses Perubahan: Jika ID ditemukan (YA), Admin dapat memasukkan data baru. Terdapat instruksi khusus di mana menekan tombol ENTER berarti data lama tetap dipertahankan (tidak diubah).
- Pembaruan & Konfirmasi: Sistem melakukan proses UpdateHero dan menampilkan pesan "Hero Berhasil Diupdate".
- Selesai: Alur berakhir dengan mengarahkan Admin kembali ke MENU ADMIN() dan prosedur ditutup (END).

1.9 Hapus Hero()



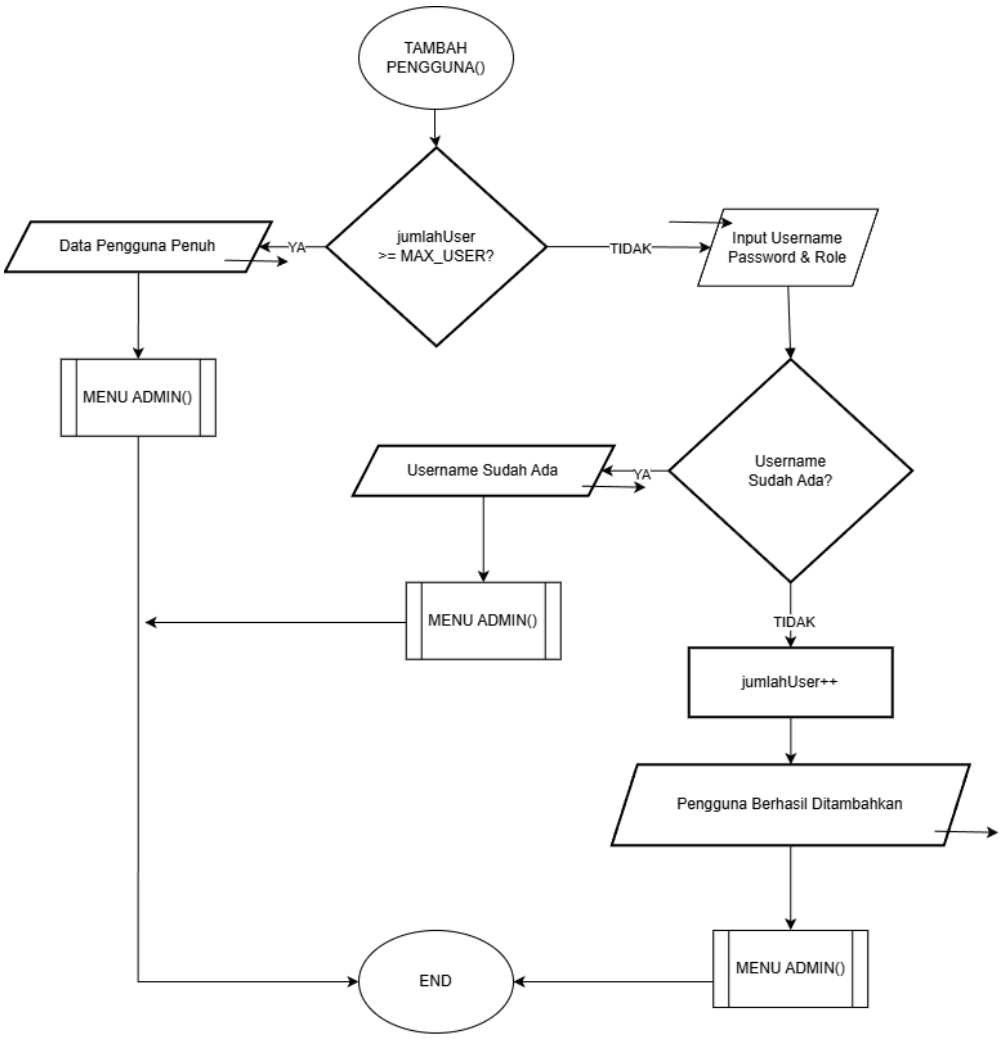
Gambar 1.9 Flowchart Hapus Hero

Penjelasan:

Flowchart ini merinci prosedur penghapusan data hero dari sistem dengan tahap verifikasi keamanan:

- Inisialisasi & Identifikasi: Program menampilkan tabel hero dan meminta Admin memasukkan ID Hero yang akan dihapus.
- Validasi ID: Sistem mengecek validitas ID. Jika TIDAK valid, muncul pesan "ID Tidak Valid" dan kembali ke MENU ADMIN().
- Konfirmasi Penghapusan: Jika ID valid, sistem meminta konfirmasi ulang (y/n).
    - Jika 'n' (TIDAK): Muncul pesan "Dibatalkan" dan kembali ke menu admin.
- Eksekusi Penghapusan: Jika pengguna memilih 'y' (YA), sistem melakukan proses Geser Array untuk merapikan data dan mengurangi hitungan total hero (jumlahHero--).
- Selesai: Setelah pesan "Hero Berhasil Dihapus" muncul, alur kembali ke MENU ADMIN() dan prosedur ditutup (END).

1.10Tambah Pengguna()



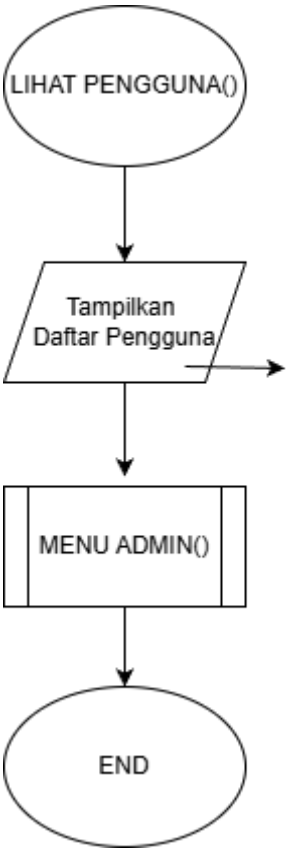
Gambar 1.10 Flowchart Tambah Pengguna

Penjelasan:

Flowchart ini merinci proses penambahan akun baru oleh Admin ke dalam database sistem:

- Validasi Kapasitas: Program memeriksa apakah jumlah pengguna sudah mencapai batas maksimal (jumlahUser >= MAX_USER). Jika penuh, muncul pesan "Data Pengguna Penuh" dan kembali ke MENU ADMIN().
- Input Data: Jika kapasitas tersedia, Admin memasukkan *Username*, *Password*, dan *Role* (peran) untuk pengguna baru.
- Validasi Duplikasi: Sistem mengecek ketersediaan *Username*. Jika sudah terdaftar, muncul pesan "Username Sudah Ada" dan kembali ke MENU ADMIN().
- Penyimpanan: Jika *Username* unik, sistem menambah hitungan total pengguna (jumlahUser++) dan menampilkan pesan "Pengguna Berhasil Ditambahkan".
- Selesai: Alur berakhir dengan kembali ke MENU ADMIN() dan prosedur ditutup (END).

1.11 Lihat Pengguna()



Gambar 1.11 Flowchart Tampilkan Pengguna

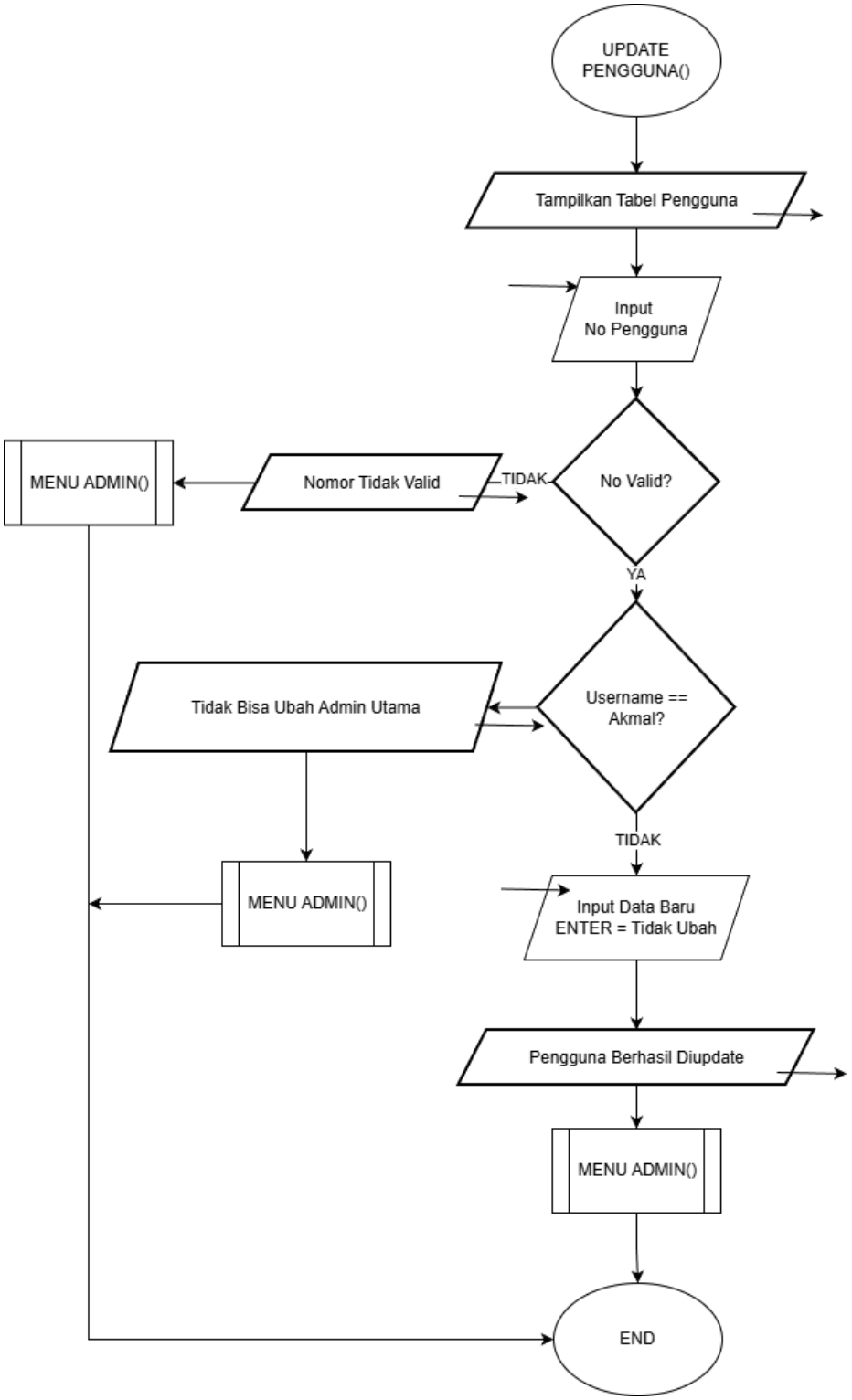
Penjelasan:

Flowchart ini menggambarkan prosedur sederhana bagi Admin untuk memantau data pengguna yang terdaftar dalam sistem:

- Inisialisasi: Program memanggil fungsi LIHAT PENGGUNA().

- Output Data: Sistem memproses dan menampilkan seluruh daftar pengguna (seperti *username* dan *role*) yang ada di dalam database ke layar.
- Navigasi & Selesai: Setelah data ditampilkan, alur kerja secara otomatis diarahkan kembali ke prosedur MENU ADMIN() dan proses dinyatakan selesai (END).

1.12 Update Pengguna()



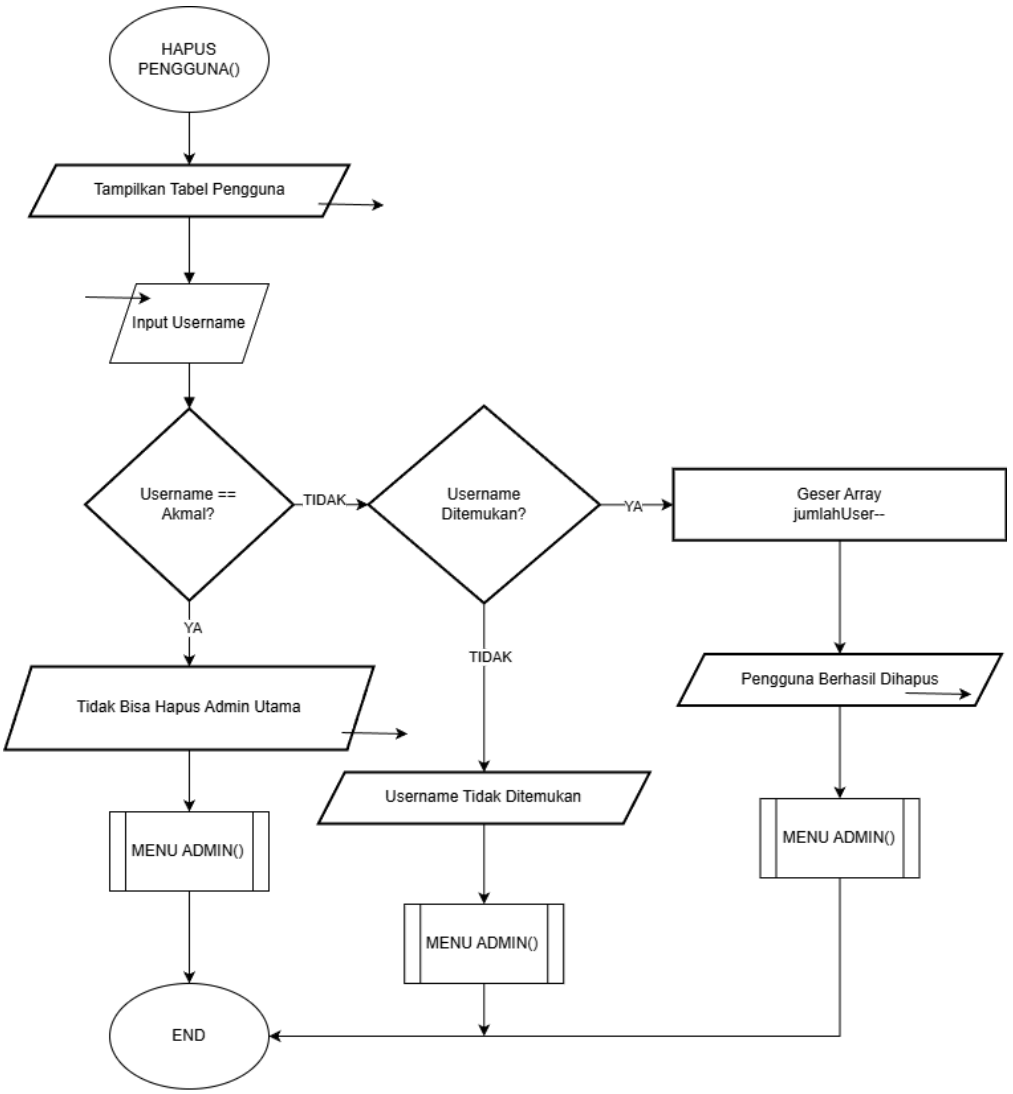
Gambar 1.12 Flowchart Update Pengguna

Penjelasan:

Flowchart ini merinci prosedur perubahan data pengguna dengan proteksi khusus untuk akun administrator utama:

- Identifikasi: Program menampilkan tabel pengguna dan meminta Admin memasukkan Nomor Pengguna yang akan diubah.
- Validasi Nomor: Sistem mengecek validitas nomor. Jika TIDAK valid, muncul pesan "Nomor Tidak Valid" dan kembali ke MENU ADMIN().
- Proteksi Admin Utama: Jika nomor valid, sistem mengecek apakah username tersebut adalah akun utama (Username == Akmal?). Jika YA, muncul pesan "Tidak Bisa Ubah Admin Utama" sebagai bentuk proteksi sistem.
- Proses Perubahan: Untuk pengguna lain, Admin dapat memasukkan data baru (opsional dengan menekan ENTER untuk tidak mengubah data).
- Konfirmasi & Selesai: Muncul pesan "Pengguna Berhasil Diupdate", lalu alur kembali ke MENU ADMIN() dan prosedur ditutup (END).

1.13 Hapus Pengguna()



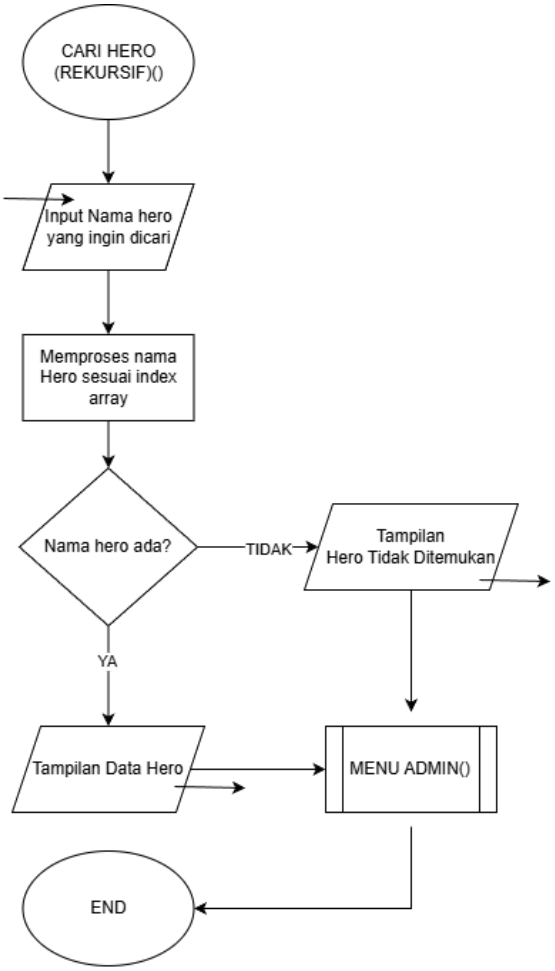
Gambar 1.13 Flowchart Hapus Pengguna

Penjelasan:

Flowchart ini merinci logika penghapusan data pengguna dengan sistem keamanan untuk melindungi akun administrator utama:

- Inisialisasi: Program menampilkan tabel pengguna yang terdaftar dan meminta Admin memasukkan Username yang ingin dihapus.
- Proteksi Akun Utama: Sistem melakukan pengecekan pertama (Username == Akmal?). Jika YA, program menampilkan pesan "Tidak Bisa Hapus Admin Utama" untuk mencegah penghapusan akun administrator inti.
- Verifikasi Keberadaan: Jika bukan akun utama, sistem mengecek apakah Username Ditemukan? dalam database.
    - Jika TIDAK, muncul pesan "Username Tidak Ditemukan".
- Eksekusi Penghapusan: Jika ditemukan (YA), sistem menjalankan proses Geser Array untuk menghapus data secara struktural dan mengurangi hitungan total pengguna (jumlahUser--).
- Selesai: Setelah konfirmasi "Pengguna Berhasil Dihapus", alur kembali ke MENU ADMIN() dan prosedur ditutup (END).

1.14 Cari Hero()



Gambar 1.14 Flowchart Cari Hero

Penjelasan:

Flowchart ini menggambarkan logika pencarian data hero menggunakan pendekatan rekursif (pemanggilan fungsi secara berulang sesuai indeks):

- Input: Admin memasukkan Nama Hero yang ingin dicari di dalam sistem.
- Proses Rekursif: Sistem memproses nama tersebut dengan menelusuri data satu per satu berdasarkan indeks array. Proses ini akan terus memanggil dirinya sendiri (rekursi) hingga data ditemukan atau semua indeks telah diperiksa.
- Percabangan Hasil:
    - YA (Ditemukan): Sistem menampilkan detail Data Hero yang dicari.
    - TIDAK (Tidak Ditemukan): Sistem memberikan pesan "Tampilan Hero Tidak Ditemukan".
- Selesai: Setelah hasil ditampilkan, alur kembali ke MENU ADMIN() dan prosedur berakhir (END).

2. Deskripsi Singkat Program

- Program Gacha Hero Mobile Legends adalah program berbasis teks yang dibuat menggunakan bahasa C untuk mensimulasikan sistem gacha hero seperti pada game Mobile Legends. Program ini mengelola data hero dan pengguna dengan menerapkan konsep pemrograman terstruktur menggunakan struct, nested struct, array of struct, fungsi, prosedur, fungsi rekursif, dan simulasi overloading.
- Program mendukung tiga jenis akses. Tamu dapat melihat daftar hero tanpa perlu login. User dapat login dan melihat daftar hero yang tersedia. Admin memiliki kendali penuh atas data hero dan pengguna melalui fitur CRUD lengkap beserta fitur pencarian hero secara rekursif.
- Sistem keamanan program menggunakan mekanisme login dengan maksimal tiga kali percobaan. Jika gagal tiga kali, program otomatis berhenti. Pengguna baru dapat mendaftar melalui fitur registrasi yang secara otomatis memberikan role sebagai user. Setelah logout, program kembali ke Menu Utama sehingga dapat digunakan bergantian oleh banyak pengguna tanpa perlu dijalankan ulang.
- Program menerapkan tiga fitur unggulan. Pertama, fungsi rekursif yang digunakan untuk mencari hero berdasarkan nama (cariHeroRekursif), menghitung jumlah hero berdasarkan role (hitungHeroDenganRole), dan menghitung faktorial. Kedua, simulasi overloading melalui tiga fungsi berlabel berbeda yaitu cetakInfo_int, cetakInfo_str, dan cetakInfo_float yang menampilkan informasi dengan format seragam namun menerima tipe data berbeda. Ketiga, nested struct melalui HeroStats yang tersimpan di dalam struct Hero untuk menyimpan data role, tier, dan win rate hero.

3. **Source Code**

A. Definisi Struct

Program mendefinisikan tiga struct utama. HeroStats menyimpan role, tier, dan winRate sebagai nested struct. Hero menyimpan data lengkap hero termasuk id, nama, lane, harga, rarity, dan HeroStats di dalamnya. User menyimpan username, password, dan role pengguna.

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX_HERO 50
#define MAX_USER 10

struct HeroStats {
    char role[20];
    char tier[5];
    int  winRate;
};

struct Hero {
    int  id;
    char nama[40];
    char lane[20];
    int  harga;
    char rarity[15];
    struct HeroStats stats;
};

struct User {
    char username[30];
    char password[20];
    char role[10];
};

struct Hero daftarHero[MAX_HERO];
int jumlahHero = 0;

struct User daftarUser[MAX_USER] = {
    {"Akmal",    "074", "admin"},
    {"Nuril",    "123", "user"},
    {"Triya",    "321", "user"},
    {"Pernanda", "111", "user"}
};
int jumlahUser = 4;
```

B. Deklarasi Maju (Forward Declaration)

Seluruh fungsi dan prosedur dideklarasikan terlebih dahulu di bagian atas sebelum main(). Hal ini diperlukan karena definisi fungsi ditulis setelah main(), sehingga compiler mengenali nama fungsi tersebut saat dipanggil di dalam main() maupun fungsi lainnya.

```
void cetakGaris(int panjang);
void cetakHeaderHero();
void cetakBarisTamu();
void cetakHeaderUser();
void inisialisasiHero();

void tampilkanSemuaHero(struct Hero daftar[], int jumlah);
void tambahHero(struct Hero daftar[], int *jumlah);
void updateHero(struct Hero daftar[], int jumlah);
void hapusHero(struct Hero daftar[], int *jumlah);

void tampilkanSemuaUser(struct User daftar[], int jumlah);
void tambahUser(struct User daftar[], int *jumlah);
void updateUser(struct User daftar[], int jumlah);
void hapusUser(struct User daftar[], int *jumlah);
void registrasi(struct User daftar[], int *jumlah);

int  login(struct User daftar[], int jumlah, int *isAdmin);
int  cariHeroRekursif(struct Hero daftar[], int idx, int jumlah, const char
*nama);
int  hitungHeroDenganRole(struct Hero daftar[], int idx, int jumlah, const
char *role);

void cetakInfo_int(const char *label, int nilai);
void cetakInfo_str(const char *label, const char *nilai);
void cetakInfo_float(const char *label, float nilai);
void menuAdmin(int userLogin);
void menuUser(int userLogin);
void menuTamu();
```

C. Menu Utama

Menu utama berjalan dalam while (pilihanAwal != 4) sehingga terus berulang sampai pengguna memilih Keluar. Di dalamnya terdapat switch dengan empat cabang: Login, Registrasi, Lihat Hero Tamu, dan Keluar.

```
int main() {
    inisialisasiHero();

    int pilihanAwal = 0;
    while (pilihanAwal != 4) {
```

```
        printf("\n");
        cetakGaris(50);
        printf("        GACHA HERO - MOBILE LEGENDS\n");
        cetakGaris(50);
        printf("  1. Login\n");
        printf("  2. Registrasi\n");
        printf("  3. Lihat Hero (Tamu)\n");
        printf("  4. Keluar\n");
        cetakGaris(50);
        printf("Pilih menu: ");
        scanf("%d", &pilihanAwal);

        switch (pilihanAwal) {
            case 1: {
                int isAdmin   = 0;
                int userLogin = login(daftarUser, jumlahUser, &isAdmin);
                if (userLogin == -1) {
                    printf("Anda telah gagal login 3 kali. Program
berhenti.\n");
                    return 0;
                }
                if (isAdmin)
                    menuAdmin(userLogin);
                else
                    menuUser(userLogin);
                break;
            }
            case 2:
                registrasi(daftarUser, &jumlahUser);
                break;
            case 3:
                menuTamu();
                break;
            case 4:
                printf("Terima kasih telah menggunakan Gacha Hero ML!\n");
                break;
            default:
                printf("Pilihan tidak valid!\n");
        }
    }
    return 0;
}
```

D. Prosedur inisialisasiHero()

Mengisi data 5 hero awal langsung ke array global daftarHero menggunakan strcpy untuk field string dan assignment langsung untuk field integer. Data yang diisi mencakup seluruh atribut hero termasuk nested struct stats.

```
void inisialisasiHero() {
    jumlahHero = 5;

    daftarHero[0].id = 1;
    strcpy(daftarHero[0].nama,       "Layla");
    strcpy(daftarHero[0].lane,       "Gold Lane");
    daftarHero[0].harga            = 150;
    strcpy(daftarHero[0].rarity,    "Epic");
    strcpy(daftarHero[0].stats.role,"Marksman");
    strcpy(daftarHero[0].stats.tier,"B");
    daftarHero[0].stats.winRate    = 51;

    daftarHero[1].id = 2;
    strcpy(daftarHero[1].nama,       "Tigreal");
    strcpy(daftarHero[1].lane,       "EXP Lane");
    daftarHero[1].harga            = 200;
    strcpy(daftarHero[1].rarity,    "Epic");
    strcpy(daftarHero[1].stats.role,"Tank");
    strcpy(daftarHero[1].stats.tier,"A");
    daftarHero[1].stats.winRate    = 54;

    daftarHero[2].id = 3;
    strcpy(daftarHero[2].nama,       "Kagura");
    strcpy(daftarHero[2].lane,       "Mid Lane");
    daftarHero[2].harga            = 600;
    strcpy(daftarHero[2].rarity,    "Legend");
    strcpy(daftarHero[2].stats.role,"Mage");
    strcpy(daftarHero[2].stats.tier,"S");
    daftarHero[2].stats.winRate    = 57;

    daftarHero[3].id = 4;
    strcpy(daftarHero[3].nama,       "Hayabusa");
    strcpy(daftarHero[3].lane,       "Jungle");
    daftarHero[3].harga            = 500;
    strcpy(daftarHero[3].rarity,    "Legend");
    strcpy(daftarHero[3].stats.role,"Assassin");
    strcpy(daftarHero[3].stats.tier,"S");
    daftarHero[3].stats.winRate    = 55;

    daftarHero[4].id = 5;
    strcpy(daftarHero[4].nama,       "Odette");
    strcpy(daftarHero[4].lane,       "Mid Lane");
    daftarHero[4].harga            = 300;
    strcpy(daftarHero[4].rarity,    "Epic");
```

```
    strcpy(daftarHero[4].stats.role,"Mage");
    strcpy(daftarHero[4].stats.tier,"A");
    daftarHero[4].stats.winRate    = 52;
}
```

E. Prosedur cetakGaris(), cetakHeaderHero(), cetakBarisTamu(), cetakHeaderUser()

Empat prosedur cetak ini bertugas menampilkan elemen antarmuka yang dipakai berulang di banyak tempat. cetakGaris() mencetak baris = sepanjang parameter panjang menggunakan loop for. Tiga prosedur lainnya mencetak header tabel hero, judul layar tamu, dan header tabel pengguna.

```
void cetakGaris(int panjang) {
    int i;
    for (i = 0; i < panjang; i++) printf("=");
    printf("\n");
}

void cetakHeaderHero() {
    printf("%-4s %-18s %-12s %-10s %-7s %-10s %-5s %s\n",
           "ID","Nama","Lane","Rarity","Harga","Role","Tier","Win%");

printf("-----------------------------------------------------------------
----\n");
}

void cetakBarisTamu() {
    printf("\n");
    cetakGaris(50);
    printf("        DAFTAR HERO GACHA (TAMU)\n");
    cetakGaris(50);
}

void cetakHeaderUser() {
    printf("%-4s %-20s %-15s %s\n","No","Username","Password","Role");
    printf("----------------------------------------\n");
}
```

F. Fungsi Rekursif cariHeroRekursif()

Menerima parameter array hero, index saat ini idx, jumlah hero, dan nama yang dicari. Memiliki dua basis case yaitu idx >= jumlah mengembalikan -1 jika tidak ditemukan, dan strcmp cocok mengembalikan idx jika ditemukan. Jika belum cocok, fungsi memanggil dirinya sendiri dengan idx + 1.

```
int cariHeroRekursif(struct Hero daftar[], int idx, int jumlah, const char
```

```
*nama) {
    if (idx >= jumlah) return -1;
    if (strcmp(daftar[idx].nama, nama) == 0) return idx;
    return cariHeroRekursif(daftar, idx + 1, jumlah, nama);
}
```

G. Fungsi Rekursif hitungHeroDenganRole()

Bekerja mirip cariHeroRekursif() namun mengembalikan akumulasi jumlah. Setiap pemanggilan menambahkan 1 jika role cocok atau 0 jika tidak, lalu ditambahkan dengan hasil rekursi index berikutnya hingga idx >= jumlah yang mengembalikan 0 sebagai basis case.

```
int hitungHeroDenganRole(struct Hero daftar[], int idx, int jumlah, const
char *role) {
    if (idx >= jumlah) return 0;
    int tambah = (strcmp(daftar[idx].stats.role, role) == 0) ? 1 : 0;
    return tambah + hitungHeroDenganRole(daftar, idx + 1, jumlah, role);
}
```

H. Fungsi login()

dalam while (kesempatan > 0). Setiap percobaan membandingkan input dengan seluruh data daftarUser menggunakan strcmp. Jika cocok, mengisi nilai *isAdmin melalui pointer dan mengembalikan index user. Jika gagal 3 kali mengembalikan -1 sehingga main() langsung return 0.

```
int login(struct User daftar[], int jumlah, int *isAdmin) {
    char username[30], password[20];
    int kesempatan = 3, i;

    while (kesempatan > 0) {
        printf("\nMasukkan Username : "); scanf("%s", username);
        printf("Masukkan Password : "); scanf("%s", password);

        for (i = 0; i < jumlah; i++) {
            if (strcmp(daftar[i].username, username) == 0 &&
                strcmp(daftar[i].password, password) == 0) {
                printf("Login berhasil! Selamat datang, %s!\n", username);
                *isAdmin = (strcmp(daftar[i].role, "admin") == 0) ? 1 : 0;
                return i;
            }
        }
        kesempatan--;
        printf("Login gagal! Kesempatan tersisa: %d\n", kesempatan);
    }
    return -1;
```

```
}
```

I. Registrasi

case 2 menangani pendaftaran akun baru. Program memeriksa apakah kapasitas pengguna sudah penuh (jumlahUser >= MAX_USER) lalu memeriksa duplikat username menggunakan perulangan for. Jika lolos, akun disimpan ke daftarUser[jumlahUser] dengan role otomatis user dan jumlahUser bertambah satu.

```
void registrasi(struct User daftar[], int *jumlah) {
    if (*jumlah >= MAX_USER) {
        printf("Kapasitas pengguna penuh!\n");
        return;
    }
    printf("\n--- Registrasi Akun Baru ---\n");
    printf("Username baru : "); scanf("%s", daftar[*jumlah].username);
    printf("Password      : "); scanf("%s", daftar[*jumlah].password);
    strcpy(daftar[*jumlah].role, "user");

    int i;
    for (i = 0; i < *jumlah; i++) {
        if (strcmp(daftar[i].username, daftar[*jumlah].username) == 0) {
            printf("Username sudah ada! Silakan pilih username lain.\n");
            return;
        }
    }
    (*jumlah)++;
    printf("Registrasi berhasil! Silakan login.\n");
}
```

J. Fungsi tampilkanSemuaHero()

Memanggil cetakHeaderHero() lalu loop for mencetak setiap baris hero. Di akhir memanggil hitungHeroDenganRole() secara rekursif untuk menampilkan jumlah hero ber-role Mage secara otomatis setiap kali tabel hero ditampilkan.

```
void tampilkanSemuaHero(struct Hero daftar[], int jumlah) {
    int i;
    printf("\nDaftar Hero:\n");

printf("-------------------------------------------------------------------
----\n");
    cetakHeaderHero();
    for (i = 0; i < jumlah; i++) {
        printf("%-4d %-18s %-12s %-10s %-7d %-10s %-5s %d%%\n",
               daftar[i].id, daftar[i].nama, daftar[i].lane,
```

```
                daftar[i].rarity, daftar[i].harga,
                daftar[i].stats.role, daftar[i].stats.tier,
                daftar[i].stats.winRate);
    }

printf("-----------------------------------------------------------------
----\n");
    printf("Total Hero: %d\n", jumlah);

    int jumlahMage = hitungHeroDenganRole(daftar, 0, jumlah, "Mage");
    printf("Jumlah hero Mage (rekursif): %d\n", jumlahMage);
}
```

K. Fungsi tambahHero()

Menerima pointer *jumlah agar jumlahHero bertambah secara global. Cek kapasitas, input seluruh field langsung ke daftar[*jumlah], lalu (*jumlah)++. Setelah tersimpan, menampilkan ringkasan hero baru menggunakan ketiga fungsi overloading cetakInfo_str(), cetakInfo_int(), dan cetakInfo_float().

```
void tambahHero(struct Hero daftar[], int *jumlah) {
    if (*jumlah >= MAX_HERO) {
        printf("Data hero sudah penuh!\n");
        return;
    }
    printf("\n--- Tambah Hero ---\n");

    struct Hero *h = &daftar[*jumlah];
    h->id = *jumlah + 1;
    printf("Nama Hero              : "); scanf(" %[^\n]",
h->nama);
    printf("Lane                   : "); scanf(" %[^\n]",
h->lane);
    printf("Rarity (Epic/Legend/Mythic): "); scanf(" %[^\n]",
h->rarity);
    printf("Harga (diamond)        : "); scanf("%d",
&h->harga);
    printf("Role (Tank/Mage/dll)   : "); scanf(" %[^\n]",
h->stats.role);
    printf("Tier (S/A/B/C)         : "); scanf(" %[^\n]",
h->stats.tier);
    printf("Win Rate (%%)           : "); scanf("%d",
```

```
&h->stats.winRate);

    (*jumlah)++;
    printf("Hero berhasil ditambahkan!\n");

    printf("\n[Info Hero Baru]\n");
    cetakInfo_str("Nama",   h->nama);
    cetakInfo_str("Role",   h->stats.role);
    cetakInfo_int("Harga",  h->harga);
    cetakInfo_int("WinRate",h->stats.winRate);
    cetakInfo_float("WR (%)", (float)h->stats.winRate);
}
```

L. Fungsi updateHero()

Memanggil tampilkanSemuaHero() terlebih dahulu, meminta input ID lalu dikurangi 1 untuk dijadikan index array. Setiap field dibaca ke buffer buf menggunakan scanf(" %[^\n]", buf). Jika strlen(buf) > 0 field diperbarui, jika kosong field tidak berubah.

```c
void updateHero(struct Hero daftar[], int jumlah) {
    tampilkanSemuaHero(daftar, jumlah);
    printf("Masukkan ID hero yang ingin diupdate: ");
    int idx; scanf("%d", &idx);
    idx--;
    if (idx < 0 || idx >= jumlah) {
        printf("ID tidak valid!\n");
        return;
    }

    struct Hero *h = &daftar[idx];
    char buf[50];
    printf("(Tekan ENTER untuk tidak mengubah)\n");

    printf("Nama  [%s]: ", h->nama);
    scanf(" %[^\n]", buf);
    if (strlen(buf) > 0) strcpy(h->nama, buf);

    printf("Lane  [%s]: ", h->lane);
    scanf(" %[^\n]", buf);
    if (strlen(buf) > 0) strcpy(h->lane, buf);

    printf("Rarity[%s]: ", h->rarity);
    scanf(" %[^\n]", buf);
    if (strlen(buf) > 0) strcpy(h->rarity, buf);

    printf("Harga [%d]: ", h->harga);
    scanf(" %[^\n]", buf);
    if (strlen(buf) > 0) h->harga = atoi(buf);

    printf("Role  [%s]: ", h->stats.role);
    scanf(" %[^\n]", buf);
    if (strlen(buf) > 0) strcpy(h->stats.role, buf);

    printf("Tier  [%s]: ", h->stats.tier);
    scanf(" %[^\n]", buf);
    if (strlen(buf) > 0) strcpy(h->stats.tier, buf);

    printf("WinRate[%d]: ", h->stats.winRate);
    scanf(" %[^\n]", buf);
    if (strlen(buf) > 0) h->stats.winRate = atoi(buf);

    printf("Hero berhasil diupdate!\n");
}
```

M. Fungsi hapusHero()

Menampilkan tabel, meminta input ID, validasi index, konfirmasi y/n. Jika dikonfirmasi, elemen array digeser ke kiri dengan loop for (i = idx; i < *jumlah - 1; i++) sambil memperbarui ID tiap elemen, lalu (*jumlah)--.

```
void hapusHero(struct Hero daftar[], int *jumlah) {
    tampilkanSemuaHero(daftar, *jumlah);
    printf("Masukkan ID hero yang ingin dihapus: ");
    int idx; scanf("%d", &idx);
    idx--;
    if (idx < 0 || idx >= *jumlah) {
        printf("ID tidak valid!\n");
        return;
    }

    struct Hero *h = &daftar[idx];
    char konfirm;
    printf("Yakin hapus '%s'? (y/n): ", h->nama);
    scanf(" %c", &konfirm);
    if (konfirm == 'y' || konfirm == 'Y') {
        int i;
        for (i = idx; i < *jumlah - 1; i++) {
            daftar[i]    = daftar[i + 1];
            daftar[i].id = i + 1;
        }
        (*jumlah)--;
        printf("Hero berhasil dihapus!\n");
    } else {
        printf("Dibatalkan.\n");
    }
}
```

N. Fungsi tambahUser(), tampilkanSemuaUser(), updateUser(), hapusUser()

Polanya sama dengan CRUD hero namun bekerja pada array daftarUser. updateUser() dan hapusUser() memiliki proteksi tambahan yang memblokir perubahan dan penghapusan pada akun admin utama Akmal menggunakan strcmp(daftar[idx].username, "Akmal").

```
void tampilkanSemuaUser(struct User daftar[], int jumlah) {
    int i;
    printf("\nDaftar Pengguna:\n");
    printf("----------------------------------------\n");
    cetakHeaderUser();
    for (i = 0; i < jumlah; i++) {
        printf("%-4d %-20s %-15s %s\n",
               i + 1, daftar[i].username, daftar[i].password,
daftar[i].role);
    }
```

```
    printf("----------------------------------------\n");
    printf("Total Pengguna: %d\n", jumlah);
}

void tambahUser(struct User daftar[], int *jumlah) {
    if (*jumlah >= MAX_USER) {
        printf("Data pengguna sudah penuh!\n");
        return;
    }
    printf("\n--- Tambah Pengguna ---\n");

    struct User *u = &daftar[*jumlah];
    printf("Username baru     : "); scanf("%s", u->username);
    printf("Password          : "); scanf("%s", u->password);
    printf("Role (admin/user) : "); scanf("%s", u->role);

    int i;
    for (i = 0; i < *jumlah; i++) {
        if (strcmp(daftar[i].username, u->username) == 0) {
            printf("Username sudah ada!\n");
            return;
        }
    }
    (*jumlah)++;
    printf("Pengguna berhasil ditambahkan!\n");
}

void updateUser(struct User daftar[], int jumlah) {
    tampilkanSemuaUser(daftar, jumlah);
    printf("Masukkan No pengguna yang ingin diupdate: ");
    int idx; scanf("%d", &idx);
    idx--;
    if (idx < 0 || idx >= jumlah) {
        printf("Nomor tidak valid!\n");
        return;
    }

    struct User *u = &daftar[idx];
    if (strcmp(u->username, "Akmal") == 0) {
        printf("Tidak bisa mengubah admin utama!\n");
        return;
    }

    char buf[50];
    printf("(Tekan ENTER untuk tidak mengubah)\n");

    printf("Username [%s]: ", u->username);
    scanf(" %[^\n]", buf);
```

```c
    if (strlen(buf) > 0) strcpy(u->username, buf);

    printf("Password [%s]: ", u->password);
    scanf(" %[^\n]", buf);
    if (strlen(buf) > 0) strcpy(u->password, buf);

    printf("Role     [%s]: ", u->role);
    scanf(" %[^\n]", buf);
    if (strlen(buf) > 0) strcpy(u->role, buf);

    printf("Pengguna berhasil diupdate!\n");
}




void hapusUser(struct User daftar[], int *jumlah) {
    tampilkanSemuaUser(daftar, *jumlah);
    char namaHapus[30];
    printf("Masukkan username yang dihapus: ");
    scanf("%s", namaHapus);

    if (strcmp(namaHapus, "Akmal") == 0) {
        printf("Tidak bisa menghapus admin utama!\n");
        return;
    }

    int idx = -1, i;
    for (i = 0; i < *jumlah; i++) {
        if (strcmp(daftar[i].username, namaHapus) == 0) { idx = i; break; }
    }

    if (idx == -1) {
        printf("Username tidak ditemukan!\n");
        return;
    }

    struct User *u = &daftar[idx];
    for (i = idx; i < *jumlah - 1; i++)
        daftar[i] = daftar[i + 1];
    (*jumlah)--;
    printf("Pengguna '%s' berhasil dihapus!\n", namaHapus);
}
```

O. Prosedur menuAdmin()

Berjalan dalam do-while (pilihan != 0) dengan 9 pilihan switch. Setiap pilihan memanggil fungsi yang sesuai dengan passing array global dan pointer jumlah sebagai

argumen. Pilihan 9 adalah demo cariHeroRekursif() yang menampilkan detail hero menggunakan fungsi overloading. Pilihan 0 keluar dari loop dan kembali ke Menu Utama.

```
void menuAdmin(int userLogin) {
    int pilihan;
    do {
        printf("\n");
        cetakGaris(50);
        printf("  Menu Admin  |  Login: %s\n",
daftarUser[userLogin].username);
        cetakGaris(50);
        printf("  1. Tambah Hero\n");
        printf("  2. Lihat Hero\n");
        printf("  3. Update Hero\n");
        printf("  4. Hapus Hero\n");
        printf("  5. Tambah Pengguna\n");
        printf("  6. Lihat Pengguna\n");
        printf("  7. Update Pengguna\n");
        printf("  8. Hapus Pengguna\n");
        printf("  9. Cari Hero (Rekursif Demo)\n");
        printf("  0. Logout\n");
        cetakGaris(50);
        printf("Pilih menu: ");
        scanf("%d", &pilihan);

        switch (pilihan) {
            case 1: tambahHero(daftarHero, &jumlahHero);          break;
            case 2: tampilkanSemuaHero(daftarHero, jumlahHero);    break;
            case 3: updateHero(daftarHero, jumlahHero);            break;
            case 4: hapusHero(daftarHero, &jumlahHero);            break;
            case 5: tambahUser(daftarUser, &jumlahUser);           break;
            case 6: tampilkanSemuaUser(daftarUser, jumlahUser);    break;
            case 7: updateUser(daftarUser, jumlahUser);            break;
            case 8: hapusUser(daftarUser, &jumlahUser);            break;
            case 9: {
                char namaCari[40];
                printf("Masukkan nama hero yang dicari: ");
                scanf(" %[^\n]", namaCari);
                int hasil = cariHeroRekursif(daftarHero, 0, jumlahHero,
namaCari);
                if (hasil == -1)
                    printf("Hero '%s' tidak ditemukan.\n", namaCari);
                else {
                    printf("Hero ditemukan di indeks %d:\n", hasil);
                    cetakInfo_str("Nama",   daftarHero[hasil].nama);
                    cetakInfo_str("Lane",   daftarHero[hasil].lane);
                    cetakInfo_str("Role",   daftarHero[hasil].stats.role);
                    cetakInfo_str("Tier",   daftarHero[hasil].stats.tier);
```

```
                    cetakInfo_int("Harga",  daftarHero[hasil].harga);

cetakInfo_int("WinRate",daftarHero[hasil].stats.winRate);
                    cetakInfo_float("WR
(%)",(float)daftarHero[hasil].stats.winRate);
                }
                break;
            }
            case 0:
                printf("Logout berhasil! Kembali ke menu utama...\n");
                break;
            default:
                printf("Pilihan tidak valid!\n");
        }
    } while (pilihan != 0);
}
```

P. Prosedur menuUser()

Berjalan dalam do-while (pilihan != 0) dengan hanya 1 pilihan aktif yaitu Lihat Hero yang memanggil tampilkanSemuaHero(). Pilihan 0 logout dan kembali ke Menu Utama.

```
void menuUser(int userLogin) {
    int pilihan;
    do {
        printf("\n");
        cetakGaris(50);
        printf("  Menu User  |  Login: %s\n",
daftarUser[userLogin].username);
        cetakGaris(50);
        printf("  1. Lihat Hero\n");
        printf("  0. Logout\n");
        cetakGaris(50);
        printf("Pilih menu: ");
        scanf("%d", &pilihan);

        switch (pilihan) {
            case 1: tampilkanSemuaHero(daftarHero, jumlahHero); break;
            case 0: printf("Logout berhasil! Kembali ke menu utama...\n");
break;
            default: printf("Pilihan tidak valid!\n");
        }
    } while (pilihan != 0);
}
```

Q. Prosedur menuTamu()

Berjalan dalam do-while (pilihanTamu != 0). Setiap iterasi memanggil cetakBarisTamu() dan cetakHeaderHero() lalu mencetak seluruh baris hero dengan loop for. Pengguna hanya bisa memilih 0 untuk kembali ke Menu Utama tanpa perlu login.

```
void menuTamu() {
    int pilihanTamu;
    do {
        cetakBarisTamu();
        cetakHeaderHero();
        int i;
        for (i = 0; i < jumlahHero; i++) {
            printf("%-4d %-18s %-12s %-10s %-7d %-10s %-5s %d%%\n",
                   daftarHero[i].id, daftarHero[i].nama, daftarHero[i].lane,
                   daftarHero[i].rarity, daftarHero[i].harga,
                   daftarHero[i].stats.role, daftarHero[i].stats.tier,
                   daftarHero[i].stats.winRate);
        }

printf("-----------------------------------------------------------------
----\n");
        printf("Total Hero: %d\n", jumlahHero);
        printf("\n  0. Kembali ke Menu Utama\n");
        printf("Pilih menu: ");
        scanf("%d", &pilihanTamu);
        if (pilihanTamu != 0) printf("Pilihan tidak valid!\n");
    } while (pilihanTamu != 0);
}
```

4. Hasil Output

```
==================================================
        GACHA HERO - MOBILE LEGENDS
==================================================
  1. Login
  2. Registrasi
  3. Lihat Hero (Tamu)
  4. Keluar
==================================================
Pilih menu:
```

Gambar 4.1 Output Menu Utama

```
Pilih menu: 1

Masukkan Username : Akmal
Masukkan Password : 074
Login berhasil! Selamat datang, Akmal!

==================================================
  Menu Admin  |  Login: Akmal
==================================================
  1. Tambah Hero
  2. Lihat Hero
  3. Update Hero
  4. Hapus Hero
  5. Tambah Pengguna
  6. Lihat Pengguna
  7. Update Pengguna
  8. Hapus Pengguna
  9. Cari Hero (Rekursif Demo)
  0. Logout
==================================================
```

Gambar 4.2 Output Login Admin dan Menu Admin

```
Pilih menu: 1

Masukkan Username : Nuril
Masukkan Password : 123
Login berhasil! Selamat datang, Nuril!

==================================================
  Menu User  |  Login: Nuril
==================================================
  1. Lihat Hero
  0. Logout
==================================================
```

Gambar 4.3 Output Login User dan Menu User

```
==================================================
Pilih menu: 1

--- Tambah Hero ---
Nama Hero                : Gusion
Lane                     : Jungler
Rarity (Epic/Legend/Mythic): Mythic
Harga (diamond)          : 100
Role (Tank/Mage/dll)     : Jungler
Tier (S/A/B/C)           : S
Win Rate (%)             : 100
Hero berhasil ditambahkan!
```

Gambar 4.4 Output Menu Admin Tambah Hero

```
Masukkan ID hero yang ingin diupdate: 6
(Tekan ENTER untuk tidak mengubah)
Nama  [Gusion]:

Gusion
Lane  [Jungler]: Mid Lane
Rarity[Mythic]: Mythic
Harga [100]: 150
Role  [Jungler]: Jungler
Tier  [S]: SS
WinRate[100]: 100
Hero berhasil diupdate!
```

Gambar 4.5 Output Menu Admin Update Hero

```
Masukkan ID hero yang ingin dihapus: 6
Yakin hapus 'Gusion'? (y/n): y
Hero berhasil dihapus!
```

Gambar 4.6 Output Menu Admin Hapus Hero

```
Pilih menu: 5

--- Tambah Pengguna ---
Username baru     : Cuki
Password          : 222
Role (admin/user) : user
Pengguna berhasil ditambahkan!
```

Gambar 4.7 Output Menu Admin Tambah Pengguna

```
==================================================
Pilih menu: 6

Daftar Pengguna:
-----------------------------------------
No   Username            Password        Role
-----------------------------------------
1    Akmal               074             admin
2    Nuril               123             user
3    Triya               321             user
4    Pernanda            111             user
5    Cuki                222             user
-----------------------------------------
Total Pengguna: 5
```

Gambar 4.8 Output Menu Admin Lihat Pengguna

```
==================================================
Pilih menu: 7

Daftar Pengguna:
-----------------------------------------
No   Username            Password        Role
-----------------------------------------
1    Akmal               074             admin
2    Nuril               123             user
3    Triya               321             user
4    Pernanda            111             user
5    Cuki                222             user
-----------------------------------------
Masukkan No pengguna yang ingin diupdate: 5
(Tekan ENTER untuk tidak mengubah)
Username [Cuki]: Cuki_Solo
Password [222]: 333
Role     [user]: user
Pengguna berhasil diupdate!
```

Gambar 4.9 Output Menu Admin Update Pengguna

```
==================================================
Pilih menu: 8

Daftar Pengguna:
-----------------------------------------
No   Username            Password        Role
-----------------------------------------
1    Akmal               074             admin
2    Nuril               123             user
3    Triya               321             user
4    Pernanda            111             user
5    Cuki_Solo           333             user
-----------------------------------------
Masukkan username yang dihapus: Cuki_Solo
Pengguna 'Cuki_Solo' berhasil dihapus!
```

Gambar 4.10 Output Menu Admin Hapus Pengguna

```
Pilih menu: 9
Masukkan nama hero yang dicari: Layla
Hero ditemukan di indeks 0:
Nama        : Layla
Lane        : Gold Lane
Role        : Marksman
Tier        : B
Harga       : 150
WinRate     : 51
WR (%)      : 51.00
```

Gambar 4.11 Output Menu Admin Cari Hero (Rekursif)

```
==================================================
Pilih menu: 0
Logout berhasil! Kembali ke menu utama...

==================================================
        GACHA HERO - MOBILE LEGENDS
==================================================
  1. Login
  2. Registrasi
  3. Lihat Hero (Tamu)
  4. Keluar
==================================================
Pilih menu:
```

Gambar 4.12 Output Menu Admin Logout

```
==================================================
        GACHA HERO - MOBILE LEGENDS
==================================================
  1. Login
  2. Registrasi
  3. Lihat Hero (Tamu)
  4. Keluar
==================================================
Pilih menu: 2

--- Registrasi Akun Baru ---
Username baru : Pria__Oslo
Password      : 111
Registrasi berhasil! Silakan login.
```

Gambar 4.13 Output Registrasi

```
Daftar Hero:
----------------------------------------------------------------------
ID   Nama              Lane        Rarity     Harga   Role       Tier  Win%
----------------------------------------------------------------------
1    Layla             Gold Lane   Epic       150     Marksman   B     51%
2    Tigreal           EXP Lane    Epic       200     Tank       A     54%
3    Kagura            Mid Lane    Legend     600     Mage       S     57%
4    Hayabusa          Jungle      Legend     500     Assassin   S     55%
5    Odette            Mid Lane    Epic       300     Mage       A     52%
----------------------------------------------------------------------
Total Hero: 5
```

Gambar 4.14 Output Daftar Hero

```
==================================================
        GACHA HERO - MOBILE LEGENDS
==================================================
  1. Login
  2. Registrasi
  3. Lihat Hero (Tamu)
  4. Keluar
==================================================
Pilih menu: 4
Terima kasih telah menggunakan Gacha Hero ML!
PS C:\Users\Acer\Documents\PRAKTIKUM APL 25>
```

Gambar 4.15 Output Fitur Keluar

```
==================================================
        GACHA HERO - MOBILE LEGENDS
==================================================
  1. Login
  2. Registrasi
  3. Lihat Hero (Tamu)
  4. Keluar
==================================================
Pilih menu: 1

Masukkan Username : PriaSolo
Masukkan Password : 111
Login gagal! Kesempatan tersisa: 2

Masukkan Username : Solofriend
Masukkan Password : 321
Login gagal! Kesempatan tersisa: 1

Masukkan Username : RatapanSolo
Masukkan Password : 000
Login gagal! Kesempatan tersisa: 0
Anda telah gagal login 3 kali. Program berhenti.
PS C:\Users\Acer\Documents\PRAKTIKUM APL 25>
```

Gambar 4.16 Output Keluar Karena Salah 3x

5. Langkah-langkah GIT

5.1 GIT Init

```
PS C:\Users\Acer\Documents\PRAKTIKUM APD\github\PRAKTIKUM_APL_B2> git init
Initialized empty Git repository in C:/Users/Acer/Documents/PRAKTIKUM APD/github/PRAKTIKUM_APL_B2/.git/
```

Gambar 5.1 GIT Init

Git Ini adalah Perintah ini digunakan untuk membuat repository Git baru di folder yang sedang kita buka. Dengan kata lain, folder biasa akan “diubah” menjadi folder yang bisa dilacak perubahannya oleh Git. Biasanya ini adalah langkah pertama

ketika kita mulai proyek baru.

5.2 GIT Add

```
hint: Disable this message with "git config set advice.addEmbeddedRepo false
PS C:\Users\Acer\Documents\PRAKTIKUM APD\github\PRAKTIKUM_APL_B2> git add .
```

Gambar 5.2 GIT Add

Kegunaan Git add adalah setelah kita mengubah atau menambahkan file, kita perlu “menandai” file itu agar siap disimpan ke Git. Perintah git add berfungsi untuk memasukkan file atau perubahan ke dalam staging area, yaitu tempat sementara sebelum benar-benar disimpan permanen ke riwayat proyek.

5.3 GIT Commit

```
PS C:\Users\Acer\Documents\PRAKTIKUM APD\github\PRAKTIKUM_APL_B2> git commit -m "frist"
[master (root-commit) 90343e2] frist
 9 files changed, 1066 insertions(+)
 create mode 100644 LATIHAN/latihan1.cpp
 create mode 100644 LATIHAN/latihan1.exe
 create mode 100644 LATIHAN/main.cpp
 create mode 100644 LATIHAN/main.exe
 create mode 100644 POSSTEST APL 1/.$2509106074_NurilAkmal_PT-1.drawio.bkp
 create mode 100644 POSSTEST APL 1/2509106074_NurilAkmal_PT-1.cpp
 create mode 100644 POSSTEST APL 1/2509106074_NurilAkmal_PT-1.drawio
 create mode 100644 POSSTEST APL 1/2509106074_NurilAkmal_PT-1.exe
 create mode 160000 PRAKTIKUM-APL-B2-25
```

Gambar 5.3 GIT Commit

Adapun Git Commit keguaannya adalah melakukan perubahan sudah ditandai dengan git add, maka git commit dipakai untuk benar-benar menyimpan perubahan itu ke riwayat proyek Git. Setiap commit biasanya disertai pesan (commit message) yang menjelaskan apa yang diubah, supaya lebih mudah dilacak di kemudian hari.

5.4 GIT Remote

```
PS C:\Users\Acer\Documents\PRAKTIKUM APD\github\PRAKTIKUM_APL_B2> git remote
origin
```

Gambar 5.4 GIT Remote

Dan adapun Git Remote keguannya adalah merepository Git yang kita buat

tadi hanya ada di komputer lokal. Supaya bisa terhubung dengan repository yang ada di internet (misalnya di GitHub, GitLab, atau Bitbucket), kita perlu menghubungkannya dengan git remote. Jadi, perintah ini semacam "menyambungkan" repository lokal dengan repository online.

5.5 GIT Push

```
PS C:\Users\Acer\Documents\PRAKTIKUM APD\github\PRAKTIKUM_APL_B2> git push
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 4 threads
Compressing objects: 100% (1/1), done.
Writing objects: 100% (2/2), 241 bytes | 80.00 KiB/s, done.
Total 2 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/elkhuwas20/PRAKTIKUM-APL-B2-25.git
   90343e2..72b4a75  master -> master
PS C:\Users\Acer\Documents\PRAKTIKUM APD\github\PRAKTIKUM_APL_B2>
```

Gambar 5.5 GIT Push

Setelah kita punya repository lokal yang sudah terhubung dengan remote, kita bisa mengirim commit yang ada di komputer ke repository online dengan git push. Dengan begitu, semua perubahan akan tersimpan di server dan bisa dilihat atau digunakan oleh orang lain (atau oleh kita sendiri di komputer lain).